

Como contribuir no site-labmim

Fluxo de trabalho, arquitetura do consumidor e contrato com o produtor micrometeorology.



Destinado a novos membros e revisores do projeto

github.com/Bruno-Mascarenhas/site-labmim

Revisao: 18/08/2026

O fluxo completo cabe em oito passos

1

Abrir issue

Contexto, resultado esperado e aceite verificavel.

2

Criar branch

Partir da main atualizada; nunca trabalhar nela.

3

Editar a fonte

Escolher o modulo correto e preservar as fronteiras.

4

Gerar e validar

Build, linters, formatacao e teste visual local.

5

Commit e push

Mudancas pequenas, relacionadas e com intencao clara.

6

Abrir PR

Explicar impacto, validacao e ligar a issue com Closes.

7

Responder a revisao

Corrigir na mesma branch e atualizar o mesmo PR.

8

Squash merge

CI verde, aprovacao de outra pessoa e branch removida.

Regra central

A main representa uma versao revisada e publicavel. Toda mudanca entra por pull request e recebe revisao independente.

Prepare o consumidor e entenda o que é fonte

1. Instale a versão fixada do Node e as dependências reproduzíveis.

```
git clone https://github.com/Bruno-Mascarenhas/site-labmim.git
cd site-labmim
nvm install && nvm use
npm ci
```

2. O build separa fontes editáveis, saídas geradas e dados operacionais.

Edite aqui

src/sites/ - identidade e conteúdo por publicação
src/template/ - estrutura compartilhada
src/territories/ - geografia
src/datasets/ - contrato de dados
site/assets/js e css - runtime compartilhado

Não edite manualmente

site/*.html, sitemap, robots e .htaccess
site/assets/css/site-theme.css
site/assets/img/
site/JSON, GeoJSON, Monitoramento, Climatologia e
Ceu
site/assets/graphs/

Por que isso importa

O próximo build sobrescreve a saída. Os dados operacionais pertencem ao pipeline Python e chegam somente no deploy; formatar ou versionar esses arquivos quebra o contrato entre repositórios.

Comece com um problema definido e uma branch curta

Issue

- Explique o que acontece hoje.
- Descreva o resultado esperado.
- Transforme o aceite em itens verificáveis.
- Registre arquivos, telas e contratos possivelmente afetados.

```
## Contexto
## Resultado esperado
## Critérios de aceite
- [ ] comportamento verificável
- [ ] evidência visual, se aplicável
```

Branch

- Atualize a main com fast-forward.
- Inclua o número da issue e uma descrição curta.
- Use feat, fix, docs ou chore.
- Mantenha um único objetivo por branch e PR.

```
git switch main
git pull --ff-only origin main
git switch -c feat/42-descricao-curta
```

Não use a main como área de trabalho

A branch principal é a linha estável. Se a mudança precisar de dois repositórios, crie branches e PRs coordenados em cada um; não misture os históricos Git.

Decida pelo alcance antes de escolher o arquivo

No consumidor site-labmim

Marca, origem, logos, redirects	<code>src/sites/<id>/identity.js</code>
Paginas, SEO, navegacao	<code>src/sites/<id>/pages.js</code>
Conteudo exclusivo	<code>src/sites/<id>/pages/</code> ou <code>fragments/</code>
Conteudo comum	<code>src/template/pages/</code>
Layout, navbar, footer	<code>src/template/layouts/</code> e <code>partials/</code>
Estado, contorno, viewport	<code>src/territories/</code>
Paths, timeline, dominios WRF	<code>src/datasets/</code>
Mapa, graficos, fetch e cache	<code>site/assets/js/</code>

Pergunta de revisao

A diferenca e editorial ou uma capacidade reutilizavel? Evite `if (site === 'ufba')` no template, renderer, CSS comum ou runtime. Declaracoes de publicacao devem permanecer declarativas.

Rode o mesmo conjunto que protege a main

```
npm run build -- --site=ufba    # desenvolvimento focado
npm run build:check             # todas as publicacoes + drift
npm run lint:all                # JS, CSS, temas, HTML e links
npm run format:check           # fonte formatada
make ci                        # conjunto completo do GitHub Actions
make serve                     # http://localhost:8000
```

O que o CI cobre

Build

descoberta, schema, referencias e saida canonica

Todas as publicacoes

build:all e bundles em dist//

Qualidade

ESLint, Stylelint, temas, icones e PurgeCSS

Documento gerado

Prettier, html-validate e links internos

Seguranca

npm audit com limite high

Revisao humana

light/dark, mobile, interacoes e console

Teste manual minimo

Abra uma pagina institucional e uma WebGIS de cada publicacao afetada. Verifique tema claro/escuro, largura mobile, dominios, linha do tempo, overlays, modais e console sem erros.

O historico deve explicar a intencao

```
git status
git add src/sites/ufba site
git commit -m "feat(site): adiciona pagina de projetos"
git push -u origin feat/42-pagina-projetos
```

Formato recomendado

tipo(escopo): descricao curta no imperativo

Exemplos: fix(webgis): corrige escala de precipitacao e docs(onboarding): atualiza contrato do produtor.

Inclua

- a fonte editada
- a saida site/ regenerada
- assets derivados pelo build
- um commit por intencao coerente
- somente arquivos relacionados a issue

Evite

- mensagens como 'updates' ou 'fix'
- misturar refactor, conteudo e dependencias
- versionar JSON/GeoJSON operacionais
- reformatar arquivos de dados
- reescrever a main ou forçar push nela

Saida gerada faz parte do PR

O build:check falha quando src/ e site/ divergem.

O PR deve permitir que outra pessoa valide

Base

Confirme que o destino é a main.

Issue

Use Closes #42 para ligar e encerrar no merge.

Mudanca

Explique o que muda e por que agora.

Impacto

Liste publicações, rotas e contratos afetados.

Validacao

Forneca comandos, URL local e passos manuais.

Evidencia

Inclua capturas quando houver alteração visual.

Modelo compacto

```
Closes #42
## O que muda
## Por que
## Como validar
- [ ] make ci
- [ ] teste visual local
## Riscos e impacto no deploy
```

PR coordenado em dois repositórios

Ligue os dois PRs na descrição, declare a ordem de deploy e mantenha o contrato compatível com a versão anterior.

A revisao fecha o ciclo na mesma branch

Quem revisa

- Confirma se o escopo corresponde a issue e se o modulo escolhido e o correto.
- Le o diff por arquivo e comenta no trecho especifico.
- Verifica CI, saida gerada, impactos cruzados e evidencias visuais.
- Aprova somente quando nao restam bloqueios.

Quem recebe comentarios

- Faz o ajuste na mesma branch.
- Roda novamente os checks proporcionais ao risco; antes do merge, roda o conjunto completo.
- Faz push; o pull request e atualizado automaticamente.
- Responde as threads e marca como resolvidas depois do ajuste.

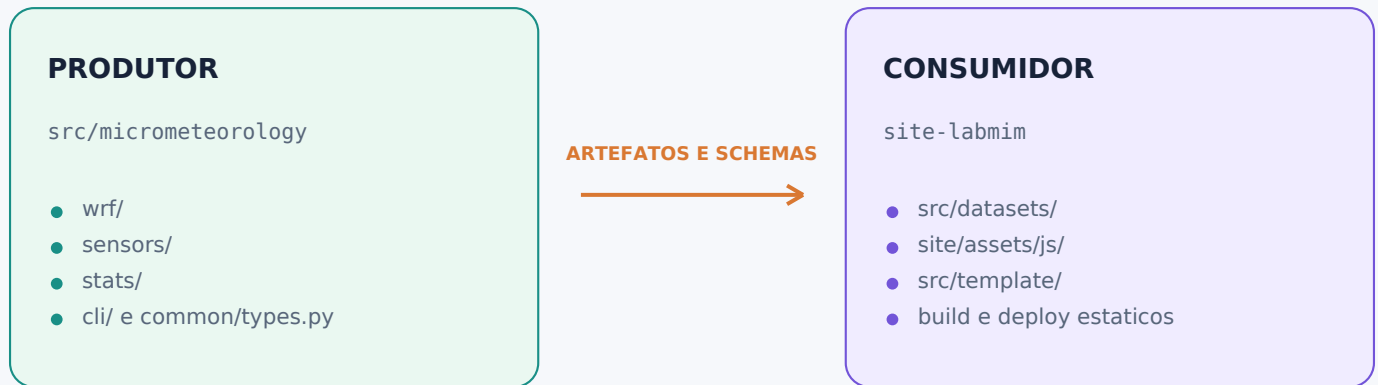
Condicao para merge

Issue ligada, escopo revisado, CI verde, teste visual concluido e aprovacao de outra pessoa. O repositorio usa squash merge; a branch e removida automaticamente depois.

Quando pedir nova revisao

Mudancas substantivas depois de uma aprovacao devem ser destacadas. Nao esconda alteracoes novas em commits de correcao sem atualizar a descricao do PR.

O produtor escreve; o consumidor publica e interpreta



Contratos ativos

WebGIS

manifest v2, grade compacta, valores, series.bin, summary.json

Overlays

WIND_VECTORS e ISOBARS, anunciados por features

Estacao

labmim-monitoring-v1 e labmim-climatology-v1

Ceu

frame, mascara, Kt x Kd e acumulada do indice de claridade

Regra de compatibilidade

Mudancas de formato devem ser aditivas. Em producao, publique primeiro o consumidor compativel e depois o produtor. O inverso pode deixar o site pedindo arquivos ou campos que ainda nao sabe ler.

Conheca os artefatos antes de alterar um contrato

Artefato do produtor	Uso no consumidor
JSON/manifest.json	timeline, versao, disponibilidade e features
GeoJSON/{D}.grid.json	grade compacta preferida
JSON/{D}_{VAR}_{NNN}.json	campo por passo; cores do mapa
JSON/{D}_ISOBARS_{NNN}.json	overlay de pressao ao nivel do mar
JSON/{D}_{VAR}.series.bin	serie de celula por HTTP Range
JSON/{D}_{VAR}.summary.json	media, minimo e maximo do dominio
Monitoramento/	janela de 7 dias em bruto, horario e WRF
Climatologia/ e Ceu/	distribuicoes, camera e produtos solares

CLIs do produtor relacionadas ao site

labmim-wrf-geojson publica o WebGIS; labmim-monitoring, labmim-climatology, labmim-sky e labmim-site-graphs publicam produtos de estacao; labmim-wrf-series mantem a serie operacional usada como camada WRF.

Deploy seguro

Site completo com .htaccess -> verificacao em producao -> pipeline atualizado -> nova rodada de dados.

Erros que costumam bloquear a revisao

Editar site/*.html

A mudanca some no proximo build.

Testar um unico site

CSS ou template comum pode quebrar a outra publicacao.

Copiar pagina compartilhada

O conteudo passa a divergir silenciosamente.

Hardcode de ID, estado ou path

O runtime deixa de ser multi-publicacao.

sourceld diferente do manifest

O cliente recebe 404 e usa cache negativo.

Tratar overlay como campo

ISOBARS e WIND_VECTORS nao geram series.bin.

Comitar dados operacionais

Arquivos grandes e privados entram no historico.

Mudar produtor primeiro

O consumidor antigo pode nao entender o novo contrato.

Ignorar light/dark e mobile

Regressoes visuais passam pelos linters.

Abrir outra branch no review

A conversa e o historico do PR se fragmentam.

Prevencao

Fronteiras declarativas, contratos versionados, build:check, make ci e inspecao das publicacoes afetadas.

Do inicio ao merge

```
git switch main
git pull --ff-only origin main
git switch -c feat/42-descricao-curta
npm run build -- --site=ufba
make ci
make serve
git add <fontes> site
git commit -m "feat(site): descricao"
git push -u origin feat/42-descricao-curta
```

Antes de pedir revisao

- | | |
|---|---|
| ☐ A issue descreve objetivo e aceite. | ☐ A branch partiu da main atualizada. |
| ☐ A alteracao esta no modulo correto. | ☐ src/ e site/ foram gerados juntos. |
| ☐ make ci passou localmente. | ☐ As publicacoes afetadas foram inspecionadas. |
| ☐ O PR explica impacto e como validar. | ☐ PRs coordenados declaram ordem de deploy. |

Referencias vivas

CONTRIBUTING.md - processo
src/sites/README.md - receitas do consumidor
Architecture.md - arquitetura completa
micrometeorology/docs/micrometeorology.md - contrato do produtor

A main deve continuar revisada, reproduzível e publicável.